

MODULEDEV · ESCAPE FROM LOCALHOST

Зона 2. Память процесса — памятка участника

Персистентное workflow-ядро



ЗОНА	Зона 2. Память процесса
ТЕХНИЧЕСКАЯ ТЕМА	Персистентное workflow-ядро
ФОРМАТ	лекция · самостоятельный проект · black-box приёмка

Результат недели

Unknown-at-build-time workflow-карта выполняется generic worker через action runtime недели 1, переживает restart и отклоняет stale completion.

Workflow и BPMN

Workflow-модель курса — самостоятельный учебный контракт, а не копия внутренней реализации банка.

flow -> process definition
step -> node
route -> transition
task -> registered action

course-1 использует BPMN-понятия как ограниченную исполняемую модель: start event → start_step, service task → automatic, message catch → wait_signal, user task → manual, end event → end, exclusive routing → переход по outcome.

BPMN XML, graphical designer, parallel gateways, timers, compensation и subprocesses не требуются.

Определение и исполнение



Workflow как данные.

Definition	Runtime state
flow_definition	process_instance
flow_version	step_instance
step_definition	workflow_job
transition_definition	task_attempt
task_definition	workflow_signal, workflow_event

Published definition неизменяема. Process навсегда хранит pinned flowVersion.

Типы шагов

Тип	Поведение
automatic	Создаёт job и вызывает зарегистрированный action
wait_signal	Долговечно ждёт идемпотентный сигнал
manual	Долговечно ждёт аудированное решение
end	Завершает технический процесс

На неделе 2 manual обязан создать persisted step и WAITING_MANUAL. Контракт ручного завершения реализуется на неделе 3.

Обязательная валидация карты

- один start;
- достижимый end;
- все steps достижимы;
- нет циклов и тупиков в обязательной части;
- action/version существует;
- worker policy достаточна;
- все action outcomes покрыты ровно одним transition;
- mapping использует JSON Pointer;
- retry policy ограничена.

task.required_policy как множество точно совпадает с policy action, а server-side scopes workflow-worker содержат это множество.

Mapping и retry

- input_mapping: target pointer action payload → source pointer process data;
- pointers соответствуют RFC 6901;
- missing source → non-retryable workflow.mapping_missing, action не вызывается;
- сформированный payload проходит request schema;
- max_attempts включает первое исполнение action;
- delays_ms содержит max_attempts - 1 значений;
- retry выполняется для retryable=true и timeout;
- mapping/contract errors не повторяются;
- истёкшая lease оставляет STALE attempt и увеличивает attempt_count, но не расходует failure budget.

Идентификаторы исполнения

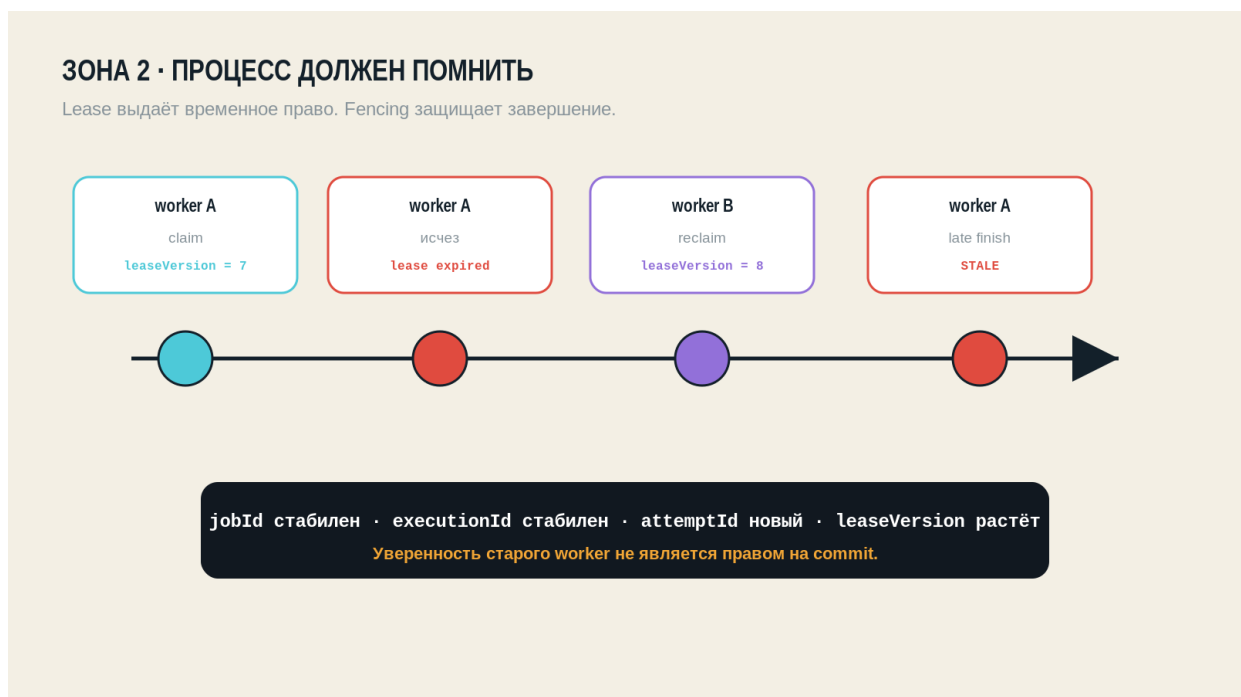
Поле	Семантика
jobId	Логическая задача

Поле	Семантика
executionId	Стабильный ключ предметного эффекта всех повторов job
attemptId	Уникальная конкретная попытка
leaseVersion	Возрастающая версия права на завершение

Retry меняет attemptId, но не jobId и не executionId.

Роль workflow_worker выполняет только workflow.claim_jobs, api.invoke, workflow.finish_job, workflow.fail_job. Прямой DML и отдельное чтение action-каталога этой роли не нужны.

Lease и fencing



Lease и fencing.

claim job → owner + leaseUntil + leaseVersion++ → execute outside claim transaction → finish only where owner and leaseVersion still match
--

Stale worker не может завершить job после reclaim.

message-id сигнала глобально уникален. Ранний сигнал объявленного pinned map типа сохраняется как ACCEPTED и применяется при входе в соответствующий wait_signal.

Crash tests используют COURSE_FAILPOINT=after_job_claim и after_action_before_finish. Checker сначала ждёт structured failpoint.reached, затем останавливает worker. Случайный sleep не является доказательством crash boundary.

Успешная транзакция automatic step

BEGIN

`api.invoke(... executionId ...)`

`validate envelope/outcome/result`

`workflow.finish_job(jobId, owner, leaseVersion, outcome, result)`

COMMIT

При error/contract violation выполняется rollback, затем отдельный fail_job записывает attempt и retry schedule.

Состояния

Process: CREATED, RUNNING, WAITING_SIGNAL, WAITING_MANUAL, COMPLETED, FAILED.

Job: READY, LEASED, RETRY_WAIT, SUCCEEDED, DEAD.

Technical FAILED не должен автоматически превращать предметную операцию в REJECTED.

Минимальная матрица тестов

- valid flow → WAITING_SIGNAL;
- hidden action + hidden map без C# changes;
- invalid graph и missing action/version;
- incomplete/duplicate outcome transitions;
- two workers, one effect;
- crash after claim → reclaim;
- stale finish rejected;
- crash between action and finish → no duplicate effect;
- retries and DEAD;
- attempts visible and append-only;
- v1 instance остаётся на v1 после activation v2;
- full worker restart сохраняет ready/delayed/waiting states.
- manual step достигает WAITING_MANUAL и переживает recreate;
- identical start возвращает прежний process, changed data даёт conflict;

- image worker не меняется после hidden publication.

Типовые ошибки

- workflow в одном C# методе;
- special branch по имени карты;
- lease без fencing condition;
- новый executionId на каждый retry;
- action effect и finish в разных транзакциях;
- история как mutable snapshot;
- published map можно обновить;
- бесконечный retry;
- worker имеет прямой DML ко всем workflow-таблицам.
- попытка реализовать полный BPMN engine вместо обязательного subset;
- sleep-based crash test без acknowledgement;
- один setup blocker размечен как несколько независимых failures.

Вопросы для самопроверки

1. Что произойдёт, если worker выполнит action и исчезнет до finish?
2. Как старый worker доказывает право завершить job?
3. Почему attemptId не подходит как idempotency key предметного action?
4. Как validator доказывает полное покрытие outcomes?
5. Где зафиксирована версия уже запущенного process?